Data storage structures

In this chapter, we focus on the organization of data stored on the underlying storage media, and how data are accessed.

Database storage architecture

Persistent data are stored on non-volatile storage. Magnetic disks as well as SSDs are block structured devices, that is, data are read or written in units of a block. In contrast, databases deal with records, which are usually much smaller than a block.

Given a set of records, the next decision lies in how to organize them in the file structure; for example, they may be stored in sorted order, in the order they are created, or in an arbitrary order.

For a CPU to access data, it must be in main memory, whereas persistent data must be resident on non-volatile storage such as magnetic disks or SSDs.

Some applications need very fast access to data and have small enough data sizes that the entire database can fit into the main memory of a database server machine. In such cases, we can keep a copy of the entire database in memory. Databases that store the entire database in memory and optimize in-memory data structures as well as query processing and other algorithms used by the database to exploit the memory residency of data are called main-memory databases.

File organization

1. A database is mapped into a number of different files that are maintained by the underlying operating system.
2. These files reside permanently on disks.
3. A file is organized logically as a sequence of records.
4. These records are mapped onto disk blocks.
5. Files are provided as a basic construct in operating systems, so we shall assume the existence of an underlying file system.

We need to consider ways of representing logical data models in terms of files.

Each file is also logically partitioned into fixed-length storage units called blocks, which are the units of both storage allocation and data transfer. Many databases allow the block size to be specified when a database instance is created. Larger block sizes can be useful in some database applications.

A block may contain several records; the exact set of records that a block contains is determined by the form of physical data organization being used. We shall assume that no record is larger than a block. Each record is entirely contained in a single block;

In a relational database, tuples of distinct relations are generally of different sizes. One approach to mapping the database to files is to use several files and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records.

❀ Fixed-length records

```
type instructor = record
                    ID varchar (5);
                    name varchar(20);
                    dept_name varchar (20);
                    salary numeric (8,2);
                  end
```

En este caso tendríamos 53 bytes por tupla

However, there are two problems with this simple approach:

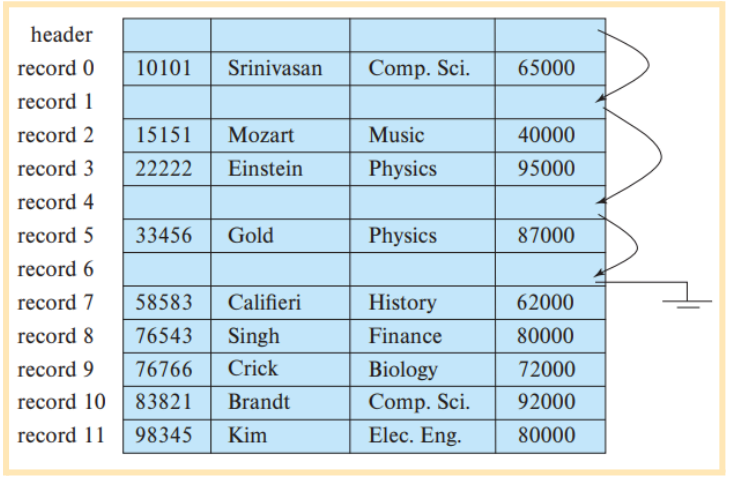
1. Unless the block size happens to be a multiple of 53 (which is unlikely), some records will cross block boundaries (two block accesses).
2. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored

To avoid the first problem, we allocate only as many records to a block as would fit entirely in the block.

When a record is deleted, we could move the record that comes after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (It might be easier to move the final record of the file into the space occupied by the deleted record).

Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a file header to store the address of the first record whose contents are deleted (and other info). We can think of these stored addresses as pointers, since they point to the location of a record.



header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

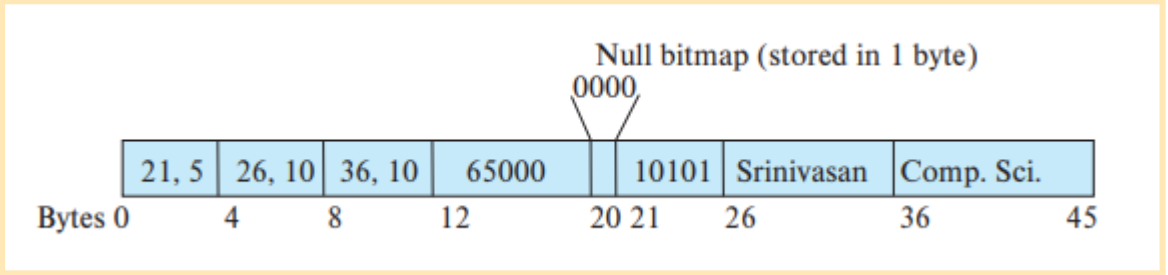
❀ Variable-length records

Variable-length records arise in database systems due to several reasons. The most common reason is the presence of variable length fields, such as strings. Different techniques for implementing variable-length records exist. Two different problems must be solved by any such technique:

1. How to represent a single record in such a way that individual attributes can be extracted easily, even if they are of variable length
2. How to store variable-length records within a block, such that records in a block can be extracted easily

The representation of a record with variable-length attributes typically has two parts: an initial part with fixed-length information, whose structure is the same for all records of the same relation, followed by the contents of variable-length attributes.

Variable-length attributes, such as varchar types, are represented in the initial part of the record by a pair (offset, length), where offset denotes where the data for that attribute begins within the record, and length is the length in bytes of the variable-sized attribute.

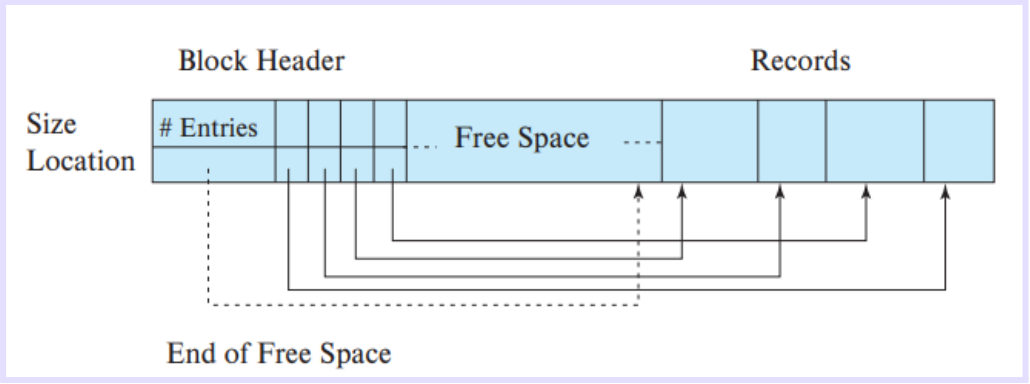


Instructor record whose first three attributes *ID, name, and dept_name* are variable-length strings, and whose fourth attribute salary is a fixed-sized number.

The figure also illustrates the use of a null bitmap, which indicates which attributes of the record have a null value. In this particular record, if the salary were null, the fourth bit of the bitmap would be set to 1, and the salary value stored in bytes 12 through 19 would be ignored.

The slotted-page structure is commonly used for organizing records within a block. There is a header at the beginning of each block, containing the following information:

➔ The number of record entries in the header
➔ The end of free space in the block
➔ An array whose entries contain the location and size of each record



The free space in the block is contiguous between the final entry in the header array and the first record. If a record is deleted, the space that it occupies is freed, and its entry is set to *deleted*.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record.

❀ **Storing large objects**

Databases often store data that can be much larger than a disk block. Many databases internally restrict the size of a record to be no larger than the size of a block.

Large objects may be stored either as files in a file system area managed by the database, or as file structures stored in and managed by the database.

There are some performance issues with storing very large objects in databases. The efficiency of accessing large objects via database interfaces is one concern. A second concern is the size of database backups. Storing large objects in the database can result in a large increase in the size of the database dumps.

Storing data in files outside the database can result in file names in the database pointing to files that do not exist, perhaps because they have been deleted, which results in a form of foreign-key constraint violation. Some databases support file system integration with the database, to ensure that constraints are satisfied, and to ensure that access authorizations are enforced.

Organization of records in files

Several of the possible ways of organizing records in files are:

- ➔ Heap file organization: Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records.

- ➔ Sequential file organization: Records are stored in sequential order, according to the value of a “search key” of each record.

- ➔ Multitable clustering file organization: Generally, a separate file or set of files is used to store the records of each relation. However, in a multitable clustering file organization, records of several different relations are stored in the same file, and in fact in the same block within a file, to reduce the cost of certain join operations.

- ➔ B+-tree file organization: The B+-tree file organization is related to the B+-tree index structure described in that chapter and can provide efficient ordered access to records even if there are a large number of insert, delete, or update operations. Further, it supports very efficient access to specific records, based on the search key.

➔ Hashing file organization: A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed.

❀ **Heap file organization**

Once placed in a particular location, the record is not usually moved. (se pone donde haya espacio)

When a record is inserted in a file, one option for choosing the location is to always add it at the end of the file. However, if records get deleted, it makes sense to use the space thus freed up to store new records. It is important for a database system to be able to efficiently find blocks that have free space, without having to sequentially search through all the blocks of the file.

Most databases use a space-efficient data structure called a free-space map to track which blocks have free space to store records. The free-space map is commonly represented by an array containing 1 entry for each block in the relation. Each entry represents a fraction f such that at least a fraction f of the space in the block is free.

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6

A. Estructura de un Array (Arreglo)

El mapa de espacio libre se representa comúnmente como un gran arreglo en el que **cada posición (índice) corresponde exactamente a un bloque del archivo de datos.** Si tu archivo tiene 16 bloques, el mapa tendrá 16 entradas. PDF + 1

B. Almacenamiento en Fracciones (f)

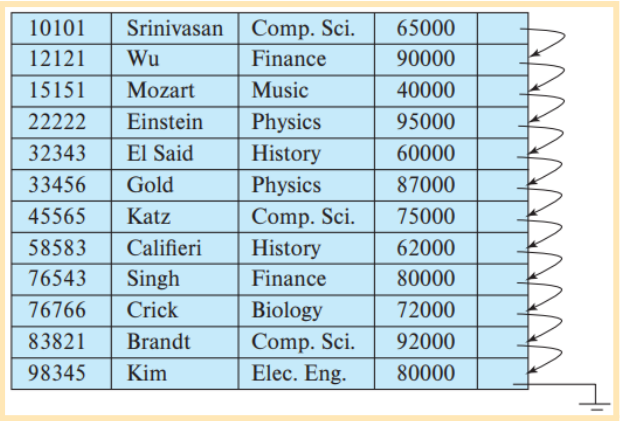
Cada entrada en este mapa no guarda el número exacto de bytes libres, sino una **fracción (f)** que representa el espacio disponible estimado. PDF

- **Ejemplo teórico del libro:** Imagina que el sistema utiliza **3 bits** para almacenar esta fracción en cada entrada. Al usar 3 bits, los números posibles van del 0 al 7, y para saber la fracción real de espacio libre, el valor de la entrada se divide por 8. PDF + 1
  - Si una entrada tiene el valor `7`, significa que al menos $\frac{7}{8}$ del bloque está libre. PDF
  - Si tiene un `4`, significa que al menos $\frac{4}{8}$ (la mitad) del bloque está libre.

The free-space map is written periodically; as a result, the free-space map on disk may be outdated, and when a database starts up, it may get outdated data about available free space.

❀ **Sequential file organization**

A sequential file is designed for efficient processing of records in sorted order based on some search key. A search key is any attribute or set of attributes; it need not be the primary key, or even a superkey.



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order.

To minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion. We can manage deletion by using pointer chains

Two rules for managing insertion:

1. Locate the record in the file that comes before the record to be inserted in searchkey order.
2. If there is a free record within the same block as this record, insert the new record there. Otherwise, insert the new record in an overflow block.

❀ Multitable Clustering File Organization

Most relational database systems store each relation in a separate file, or a separate set of files. Thus, each file, and as a result, each block, stores records of only one relation, in such a design. However, in some cases it can be useful to store records of more than one relation in a single block.

This structure allows for efficient processing of the join (between two relations).

A multitable clustering file organization is a file organization that stores related records of two or more relations in each block. The cluster key is the attribute that defines which records are stored together; in our preceding example, the cluster key is dept_name.

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

❀ **Partitioning**

Many databases allow the records in a relation to be partitioned into smaller relations that are stored separately. Such table partitioning is typically done on the basis of an attribute value.

The cost of some operations, such as finding free space for a record, increases with relation size; by reducing the size of each relation, partitioning helps reduce such overheads.

Data-dictionary storage

In general, such “data about data” are referred to as metadata. Relational schemas and other metadata about relations are stored in a structure called the data dictionary or system catalog. We store:

- ➔ Names of the relations
- ➔ Names of the attributes of each relation
- ➔ Domains and lengths of attributes
- ➔ Names of views defined on the database, and definitions of those views
- ➔ Integrity constraints
- ➔ Names of users, the default schemas of the users, and passwords or other information to authenticate users
- ➔ Information about authorizations for each user DATA ON USERS OF THE SYSTEM

The data dictionary may also note the storage organization of relations.

También almacena donde están almacenadas las relaciones, información sobre los índices de cada relación.

All this metadata information constitutes a miniature database. By using database relations to store system metadata, we simplify the overall structure of the system and harness the full power of the database for fast access to system data. The exact choice of how to represent system metadata by relations must be made by the system designers.

Database buffer

Since data access from disk is much slower than in-memory data access, a major goal of the database system is to minimize the number of block transfers between the disk and memory.

One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The buffer is that part of main memory available for storage of copies of disk blocks. The subsystem responsible for the allocation of buffer space is called the buffer manager.

❀ Buffer manager

Programs in a database system make requests on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block.

Replacement strategy

When there is no room left in the buffer, a block must be evicted, that is, removed, from the buffer before a new one can be read in. Most operating systems use a least recently used (LRU) scheme.

Pinned blocks

The process executes a pin operation on the block; the buffer manager never evicts a pinned block. When it has finished reading data, the process should execute an unpin operation, allowing the block to be evicted when required.

The block cannot be evicted until all processes that have executed a pin have then executed an unpin operation. A simple way to ensure this property is to keep a pin count for each buffer block.

Shared and exclusive locks on buffer

The locking system provided by the buffer manager allows a database process to lock a buffer block either in shared mode or in exclusive mode before accessing the block, and to release the lock later, after the access is completed.

Output of blocks

It makes sense to not wait until the buffer space is needed, but to rather write out (output) updated blocks ahead of such a need. Then, when space is required in the buffer, a block that has already been written out can be evicted.

Forced output of blocks

There are situations in which it is necessary to write a block to disk, to ensure that certain data on disk are in a consistent state. Such a write is called a forced output of a block.

❀ Buffer replacement strategies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. It is not possible to predict accurately which blocks will be referenced. If a block must be replaced, the least recently referenced block is replaced. This approach is called the least recently used (LRU) block-replacement scheme.

```
select *
from instructor natural join department;
```

The buffer manager should be instructed to free the space occupied by an instructor block as soon as the final tuple has been processed. This buffer-management strategy is called the toss-immediate strategy. For some systems, the optimal strategy for block replacement for the above procedure is the most recently used (MRU) strategy.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again.

Reordering of writes and recovery

Database buffers allow writes to be performed in-memory and output to disk at a later time, possibly in an order different from the order in which the writes were performed. Such reordering can lead to inconsistent data on disk in the event of a system crash.

However, most disks do not come with a non-volatile write buffer; instead, modern file systems assign a disk for storing a log of the writes in the order that they are performed. Such a disk is called a log disk.

For each write, the log contains the block number to be written to, and the data to be written, in the order in which the writes were performed. File systems that support log disks as above are called journaling file systems.

Column oriented storage

In contrast, in column-oriented storage, also called a columnar storage, each attribute of a relation is stored separately, with values of the attribute from successive tuples stored at successive positions in the file.

In the simplest form of column-oriented storage, each attribute is stored in a separate file.

Column-oriented storage thus has the drawback that fetching multiple attributes of a single tuple requires multiple I/O operations.

However, column-oriented storage is well suited for data analysis queries, which process many rows of a relation, but often only access some of the attributes. The reasons are as follows:

- ➔ Reduced I/O: When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory.
- ➔ Improved CPU cache performance: When the query processor fetches the contents of a particular attribute, with modern CPU architectures multiple consecutive bytes, called a cache line, are fetched from memory to CPU cache.

- ➔ Improved compression: Storing values of the same type together significantly increases the effectiveness of compression, when compared to compressing data stored in row format.
- ➔ Vector processing: Many modern CPU architectures support vector processing, which allows a CPU operation to be applied in parallel on a number of elements of an array.

Databases that use column-oriented storage are referred to as column stores, while databases that use row-oriented storage are referred to as row stores.
Drawbacks: Cost of tuple reconstruction, Cost of tuple deletion and update, Cost of decompression.

A sequence of tuples occupying several hundred megabytes is broken up into a columnar representation called a stripe.

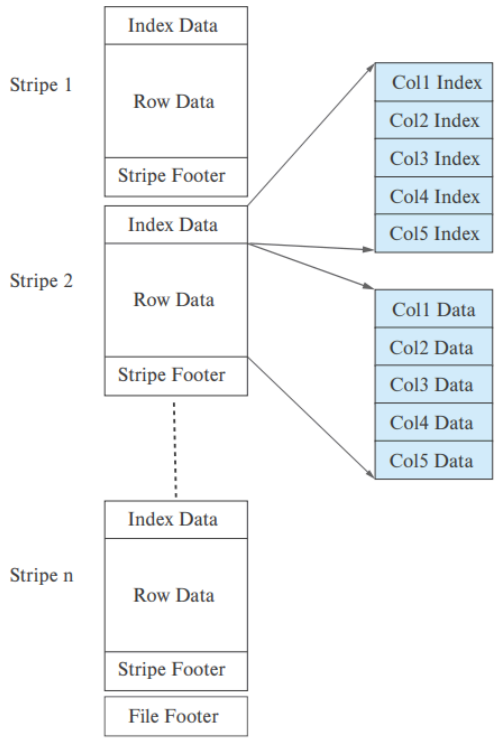


Figure 13.15 Columnar data representation in the ORC file format.

Some column-store systems allow groups of columns that are often accessed together to be stored together, instead of breaking up each column into a different file.

The choice of which attributes to store together depends on the query workload.

Using column-oriented storage within the block provides the benefits of more efficient memory access and cache usage, as well as the potential for using vector processing on the data.

Storage organization in main-memory databases

Today, main-memory sizes are large enough, and main memory is cheap enough, that many organizational databases fit entirely in memory.

Such large main memories can be used by allocating a large amount of memory to the database buffer, which will allow the entire database to be loaded into buffer, avoiding disk I/O operations for reading data; updated blocks still have to be written back to disk for persistence.
A main-memory database is a database where all data reside in memory.

In contrast, in a main-memory database, it is possible to keep direct pointers to records in memory, and accessing a record is just an in-memory pointer traversal, which is a very efficient operation. This is possible as long as records are not moved around.

Many main-memory databases do not use a slotted-page structure for allocating records. Instead they directly allocate records in main memory, and ensure that records never get moved due to updates to other records.

However, a problem with direct allocation of records is that memory may get fragmented if variable sized records are repeatedly inserted and deleted. The database must ensure that main memory does not get fragmented over time.

For a column-oriented storage scheme the logical array for a column may be divided into multiple physical arrays. An indirection table stores pointers to all the physical arrays.